IQRA UNIVERSITY

DEPARTMENT OF COMPUTER AND TECHNOLOGY

OBJECT ORIENTED PROGRAMMING

ASSIGNMENT 1

Name: Adil Ali

Submitted to: Sir Abdul Baqi

Registration Number: 052-26-128036

Date: 06/20/2026

Program: BS Artificial Intelligence

Semester: 2nd

Date:

Objected Oriented Programming:.

Object-oriented programming (OOP) is a programming approach or style or paradigm that organizes software design arround data and objects.

Important points:-

- OOP combines data and functions into a single unit called a class.
- It helps solve real-world problems in a simple and organized way.
- OOP is mainly based on classes and objects.
- It includes important concepts such as inheritance, encapsulations, and polymorphism.
- Another important concept of OOP is abstraction.
- It focuses on representing real-world thing using classes and objects.
- OOP follows a bottom-up approach.
- It provides data protection through encapsulation by keeping private data safe.

Date:

* Inheritance allows programmers to reuse existing code instead of writing everything again
* OOP makes programs easier to manage, update, and maintain.

Example:

```
#include<iostream>
Using namespace std;
class Adil {
Public:
    string = "name";
    int = age;
    double = gPA, ID;
    float = gPA;
    void display (){
     cout<< "Name:"<<name<<endl;
     cout<<"Age:"<<age<<endl;
     cout<<"ID:"<<ID<<endl;
     cout<<"GPA:"<<gPA<<endl;
     }
};
  int main (){
     Adil a1;
     a1.name = "Adil";
     a1.age = 21;
     a1.ID = sp-26-128036;
     a1.gPA = 3.0;
     a1.display ();
     return 0;
}
```

Date:

1. Functional Programming :-

Definition :

Functional programming is a programming approach that uses functions to solve problems. It focuses on pure functions and avoids changing data unnecessarily.

Main points :-

1. It uses functions to perform different tasks
2. Data and functions are handled separately
3. It focuses on function-based problem Solving
4. It encourage code reusability.
5. Pure functions produce the same output for the same input.

Functions Defined Inside a class :-

Definition :- A function defined directly inside a class is called a member function.

* It automatically acts as a member function, having direct access to all private and public data members of that class.

Example :-

```
#include <iostream>
using namespace std;
```

Date:

```
class Adil {
  public :
    void display () {
      cout << "Hello, I am Adil";
    }
};
int main () {
  Adil obj;
  obj.display();
  return 0;
}
```

Output :-

Hello, I am Adil.

Function defined outside the class :- When we declare a member function inside the class, but define its complete body the class function.

* We use scope resolution operator (::) to define the function out the class.

```
#include<iostream>
using namespace std;
class Adil {
  Public :
    void display();
};
void Adil :: display() {
  cout << "Hello, I am Adil" << endl;
}
```

Date:

```
int main () {
    Adil data ;
    data display ();
    return 0;
}
```

Bank Management System:.

Access Specifiers are the important concepts in object oriented programming which mainly control the accessibility of data members and member functions in a class. In a Bank Management system, access specifiers help protect information which are sensitive such as account numbers and account balance.

Private :-

private members can only be accessed inside the same class.

- Account number and balance should be private.
- It protects data from unauthorized access.
- Private data cannot be accessed directly from outside the class.

Public :-

Public members can be accessed from outside the class.

Date:

→ Functions like deposit(), withdraw(), and getBalance() can be declared public.

Protected :-

protected members can be accessed within the same class and its child classes. They can be used while creating Saving Account and current Account classes.

* Access Specifiers provides security and controlled access to bank account classes.

Encapsulation :-

Encapsulation is the process of binding data members and member functions into a single unit called class, while restricting the directed access to the internal data.

* It protects data from direct access.
* It can be made by declaring data members private.
* We use getter and setter functions for controlled access.

Date:

```
#include <iostream>
using namespace std;
class BankAccount {
private:
    int accountnumber;
    double balance;
public:
    BankAccount(int acc, double
                bal) {
    accountNumber = acc;
    balance = bal;
}
double getBalance() {
    return balance;
}
void setBalance(double amount){
    if (amount>=0) {
        balance = amount;
    }
}
};
int main() {
    BankAccount b1(101, 10000)
    cout<<"Balance:"<<b1.get-
    Balance();
    b1.setBalance(9000);
    cout<<"Balance updated:"<<
    b1.getBalance();
    return 0;
}
```